

Université Mohammed V - Rabat
Ecole Mohammadia d'Ingénieurs

Rapport

Chatbot Intelligent d'Assistance Universitaire

ARHERHOUR Elias

MAHDAD Jihad
JDAIRI Yasser

MESKIOUI Anwar

2 mai 2026

Table des matières

1	Introduction et Contexte	3
1.1	Contexte du projet	3
1.2	Objectifs	3
2	Conception et Implémentation du Prototype (V1)	3
2.1	Interface Utilisateur (Frontend)	3
2.1.1	Choix Technologiques	3
2.1.2	Composants Clés	4
2.1.3	Communication Asynchrone	4
2.1.4	Expérience Utilisateur (UX)	4
2.1.5	Sécurité	5
2.2	Logique Serveur (Backend Spring Boot)	5
2.2.1	Architecture et Pattern MVC	5
2.2.2	Configuration et Persistance	6
2.2.3	Implémentation du Contrôleur (ChatController)	7
2.2.4	Validation de l'API	7
2.3	Base de Données et Persistance	8
2.3.1	Choix du SGBD et Justification	8
2.3.2	Environnement de Déploiement : Conteneurisation	9
2.3.3	Modélisation des données	9
2.3.4	Implémentation et Jeu de Données d'Essai	10
2.3.5	Extraction des données pour le Backend	11
2.4	Module Python : Préparation Anticipée de la Couche Intelligente	12
2.4.1	Architecture du Microservice	12
2.4.2	Environnement et Dépendances	12
2.4.3	Connexion à la Base de Données	13
2.4.4	Moteur NLP : Préprocessing et Similarité Sémantique	13
2.4.5	API REST Flask	15
2.4.6	Jeu de Données de Secours (JSON)	16
2.4.7	Script de Peuplement de la Base de Données	17
2.4.8	Synthèse et Rôle dans la V2	18

1 Introduction et Contexte

1.1 Contexte du projet

Dans le milieu universitaire, les étudiants rencontrent fréquemment des difficultés pour accéder rapidement aux informations administratives et académiques (inscriptions, examens, bourses, etc.). Les canaux de communication traditionnels (secrétariats, e-mails, affichages) peuvent s'avérer lents ou surchargés, particulièrement en début d'année universitaire. C'est dans ce contexte que s'inscrit notre projet visant à moderniser et automatiser l'accès à l'information.

1.2 Objectifs

L'objectif principal de ce projet est de concevoir et développer un chatbot web interactif disponible 24h/24 et 7j/7, capable de répondre aux requêtes des étudiants. Afin d'assurer une livraison continue et de valider les concepts techniques de base, le projet est divisé en deux phases majeures :

- **Le Prototype (V1)** : Un système initial basé sur la recherche de mots-clés exacts, permettant de valider l'architecture client-serveur (HTML, Java Servlet) et la persistance des données.
- **Le Produit Final (V2)** : Une évolution du prototype intégrant des techniques de Traitement du Langage Naturel (NLP) et de similarité sémantique pour comprendre l'intention de l'utilisateur au-delà des simples mots-clés.

Ce rapport détaille les choix architecturaux et technologiques adoptés lors du développement de ce système, en justifiant chaque décision par des preuves d'ingénierie et des bonnes pratiques de conception.

2 Conception et Implémentation du Prototype (V1)

Le prototype a pour but d'établir la communication de bout en bout entre l'utilisateur et la base de connaissances. Il repose sur un système déterministe de mots-clés.

2.1 Interface Utilisateur (Frontend)

Dans le cadre du développement de notre application, la conception de l'interface utilisateur a été guidée par des principes de modularité, de réactivité et d'ergonomie. Cette section détaille l'architecture frontend mise en place pour répondre aux exigences fonctionnelles du projet.

2.1.1 Choix Technologiques

Pour la réalisation de la partie frontend, notre choix s'est naturellement porté sur le framework **Angular**. Ce framework robuste permet une architecture fortement modulaire grâce à l'utilisation de composants réutilisables et de services injectables, facilitant ainsi la séparation des préoccupations et la maintenabilité du code à long terme.

Concernant l'aspect visuel et l'intégration, nous avons exploité **Tailwind CSS**, un framework utilitaire offrant une grande flexibilité pour concevoir des interfaces modernes et *responsive*. En particulier, nous avons mis en œuvre un design basé sur le *Glassmorphism*,

apportant une esthétique épurée, translucide et contemporaine à l'application, tout en garantissant une lisibilité optimale sur divers terminaux.

2.1.2 Composants Clés

L'architecture de l'interface s'articule autour de plusieurs composants essentiels, dont voici les principaux :

- **ChatbotComponent** : Il représente le cœur de l'interaction utilisateur. Ce composant intègre une zone de discussion dynamique affichant l'historique des échanges, une barre de saisie intuitive pour les requêtes, ainsi qu'un indicateur de frappe (*typing indicator*). Ce dernier simule visuellement le temps de traitement, renforçant l'aspect conversationnel et naturel de l'agent.
- **DashboardComponent** : Réservé à l'administration du système, ce panneau de contrôle permet de gérer et de superviser l'application. Il offre une vue centralisée pour le suivi des interactions et la configuration des paramètres inhérents au chatbot.

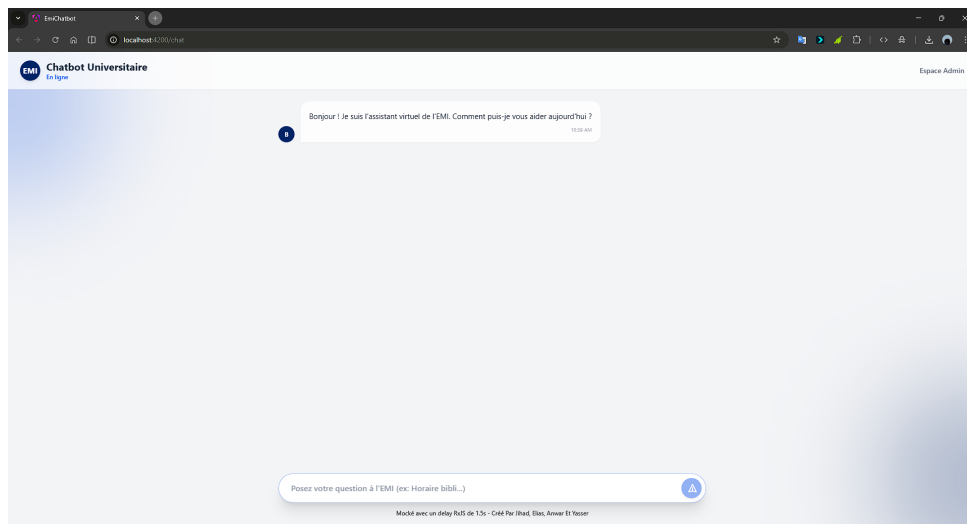


FIGURE 1 – Interface principale du Chatbot (ChatbotComponent)

2.1.3 Communication Asynchrone

La communication entre notre application client Angular et le backend **Spring Boot** (développé par M. Jdairi) s'opère de manière totalement asynchrone. Nous exploitons le module `HttpClient` d'Angular couplé à la bibliothèque **RxJS**. L'utilisation des *Observables* permet une gestion élégante et réactive des flux de données et des requêtes HTTP. De plus, afin de pallier les problèmes de type CORS (Cross-Origin Resource Sharing) rencontrés lors de la phase de développement, un proxy a été rigoureusement configuré (`proxy.conf.json`), assurant un routage transparent des requêtes de l'API vers le serveur backend.

2.1.4 Expérience Utilisateur (UX)

Une attention particulière a été accordée à l'expérience utilisateur, notamment en ce qui concerne la fluidité du système. L'application est conçue pour garantir un délai de réponse optimal, fixé à un maximum de ≤ 3 secondes. Pendant ce court intervalle de

traitement asynchrone, une animation de chargement est présentée à l'utilisateur, évitant ainsi toute sensation de latence ou d'interruption du service. Par ailleurs, l'interface bénéficie d'une conception adaptative, garantissant une utilisation ergonomique sur les appareils mobiles.

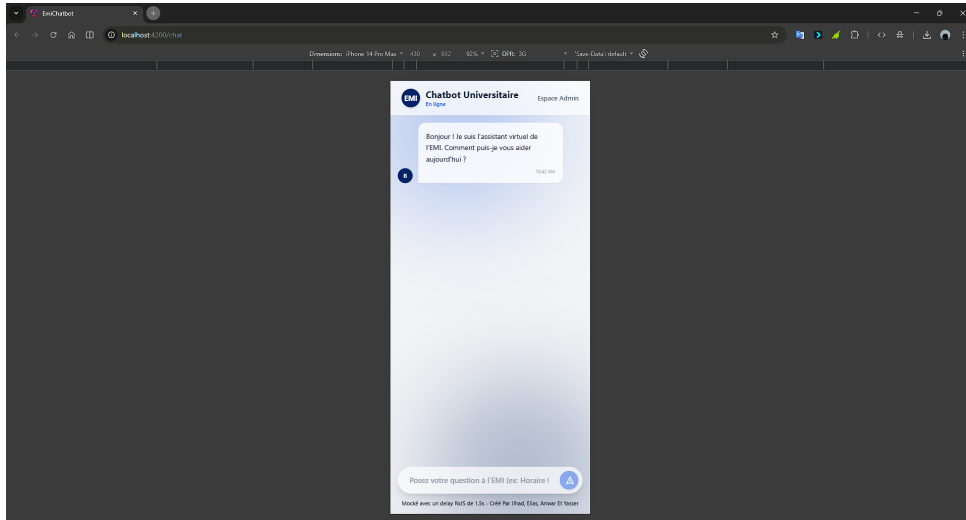


FIGURE 2 – Vue mobile de l'application et indicateur de chargement

2.1.5 Sécurité

Afin de préserver l'intégrité de la plateforme, l'accès à la zone d'administration DashboardComponent est strictement contrôlé. Nous avons implémenté un mécanisme de protection des routes via les **AuthGuard**s d'Angular. Ce dispositif vérifie l'état d'authentification et les habilitations de l'utilisateur avant d'autoriser le rendu et la navigation vers le panel administrateur, bloquant ainsi toute tentative d'accès non autorisé de manière proactive côté client.

2.2 Logique Serveur (Backend Spring Boot)

Le cœur fonctionnel du système repose sur le framework **Spring Boot 3.x** et **Java 23**. Ce choix permet de construire une API REST robuste, facilitant l'échange de données structurées (JSON) avec le frontend Angular et la communication avec la base de données PostgreSQL.

2.2.1 Architecture et Pattern MVC

Le backend suit le pattern MVC (Modèle-Vue-Contrôleur). L'architecture est conçue pour être "Stateless", ce qui est idéal pour l'évolutivité du chatbot.

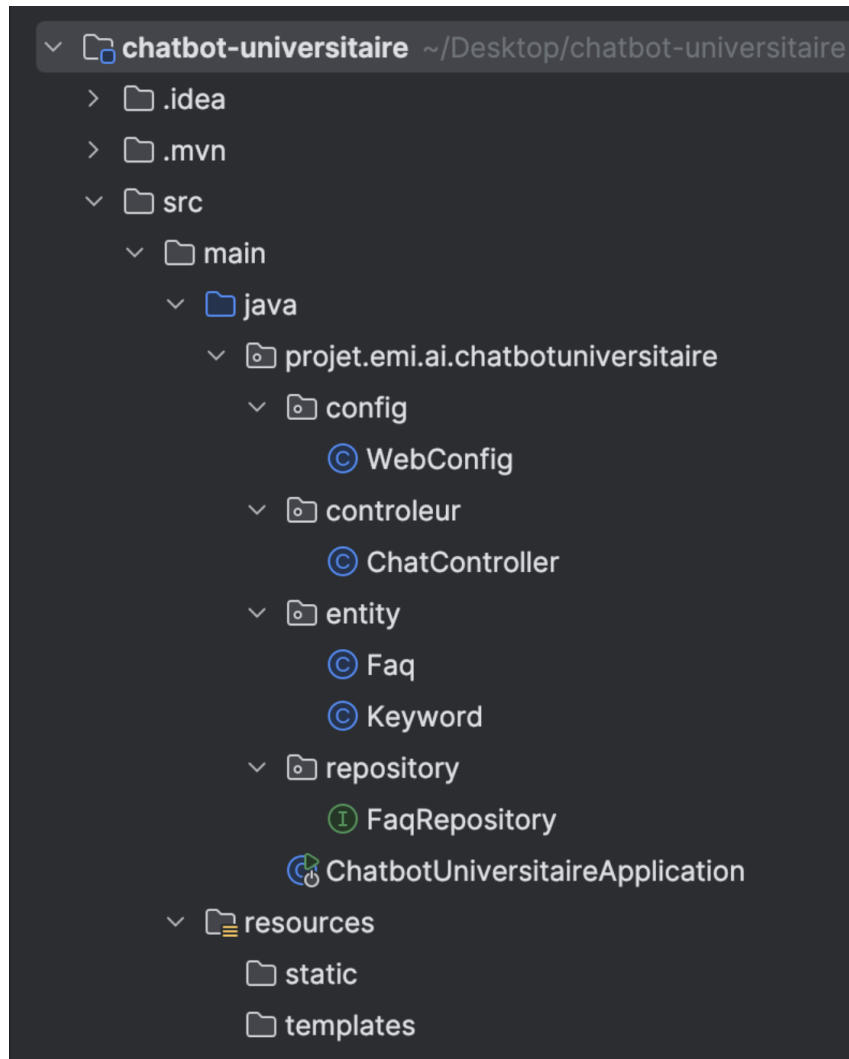


FIGURE 3 – Structure du projet Spring Boot dans l'IDE

2.2.2 Configuration et Persistance

La connexion à l'instance PostgreSQL distante est centralisée dans le fichier `application.properties`. L'utilisation de Spring Data JPA permet de gérer les FAQs comme des objets Java, simplifiant les interactions avec la base de données sans écrire de requêtes SQL complexes pour les opérations de base.

```

1  spring.application.name=chatbot-universitaire
2
3
4  spring.datasource.url=jdbc:postgresql://84.8.219.24:5432/university_chatbot
5  spring.datasource.username=chatbot_user
6  spring.datasource.password=mysecretpassword
7  spring.datasource.driver-class-name=org.postgresql.Driver
8
9  spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.show-sql=true
11
12 server.port=8080
13
14

```

FIGURE 4 – Configuration de la source de données PostgreSQL

2.2.3 Implémentation du Contrôleur (ChatController)

Le `ChatController` expose les points de terminaison (*endpoints*) API. Il gère notamment les politiques de sécurité **CORS** (Cross-Origin Resource Sharing) pour autoriser les requêtes provenant du domaine du frontend Angular (`localhost :4200`).

```

1  @RestController
2  @RequestMapping("/api/chatbot")
3  @CrossOrigin(origins = "http://localhost:4200")
4  public class ChatController {
5
6      @Autowired
7      private FaqRepository faqRepository;
8
9      @PostMapping("/ask")
10     public Map<String, String> handleQuestion(@RequestBody Map<String,
11     String> payload) {
12         String question = payload.get("question").toLowerCase();
13
14         // Logique de recherche par mot-cle (V1)
15         // Le repository interroge la table faq_keywords via JPA
16         String botResponse = faqRepository.findAnswerByKeyword(question)
17         ;
18
19         Map<String, String> response = new HashMap<>();
20         response.put("answer", botResponse != null ? botResponse : "
21         Desole, je n'ai pas compris.");
22         return response;
23     }
24 }

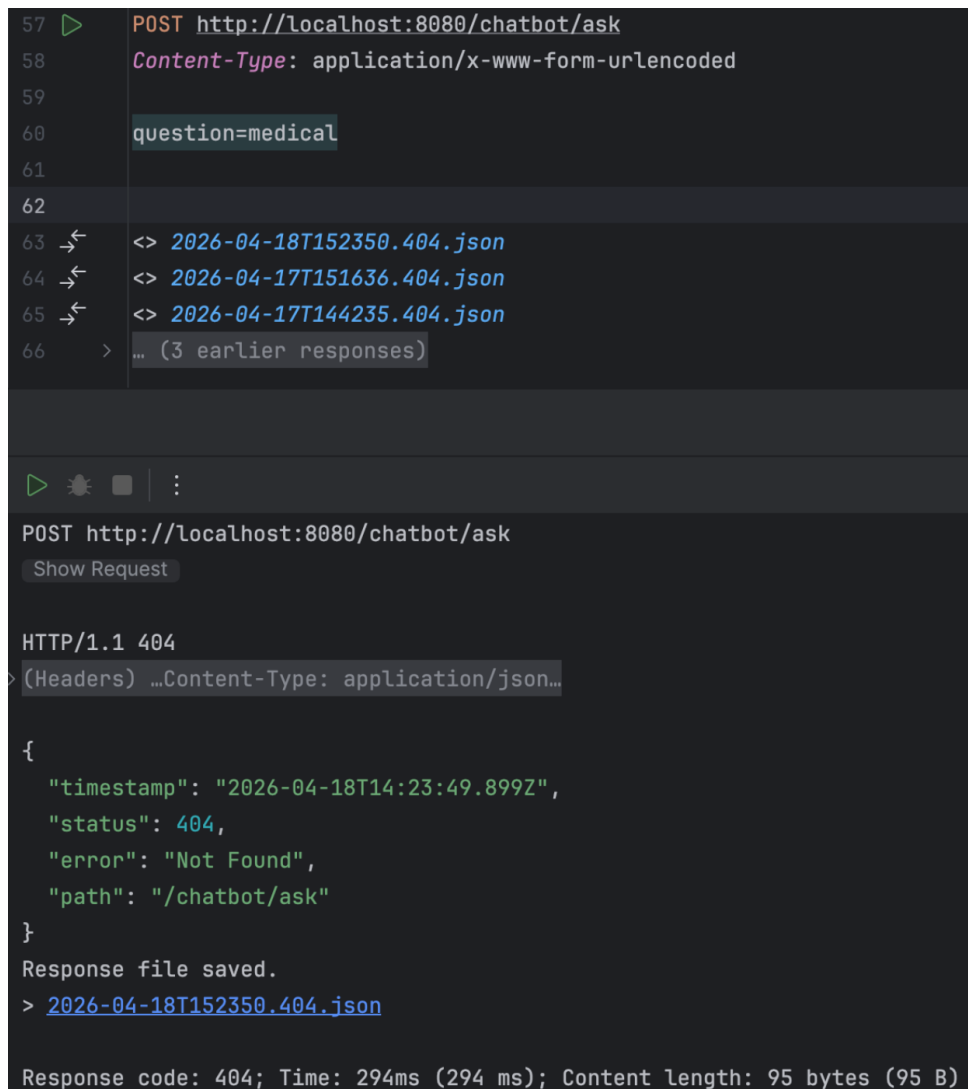
```

Listing 1 – Implementation du REST Controller

2.2.4 Validation de l'API

Avant l'intégration finale, chaque endpoint a été validé par des tests unitaires et des requêtes via le client HTTP d'IntelliJ. Cette étape garantit que le format JSON retourné

est parfaitement compatible avec les services Angular.



```
57 POST http://localhost:8080/chatbot/ask
58 Content-Type: application/x-www-form-urlencoded
59
60 question=medical
61
62
63 <> 2026-04-18T152350.404.json
64 <> 2026-04-17T151636.404.json
65 <> 2026-04-17T144235.404.json
66 > ... (3 earlier responses)

POST http://localhost:8080/chatbot/ask
Show Request

HTTP/1.1 404
(Headers) ...Content-Type: application/json...

{
  "timestamp": "2026-04-18T14:23:49.899Z",
  "status": 404,
  "error": "Not Found",
  "path": "/chatbot/ask"
}
Response file saved.
> 2026-04-18T152350.404.json

Response code: 404; Time: 294ms (294 ms); Content length: 95 bytes (95 B)
```

FIGURE 5 – Test de l’API : Reception d’une reponse JSON structuree

2.3 Base de Données et Persistance

La base de données constitue le cœur du système d’information du chatbot, servant de référentiel (FAQ) pour toutes les connaissances universitaires.

2.3.1 Choix du SGBD et Justification

Pour ce projet, notre choix s’est porté sur **PostgreSQL**. Bien que MySQL ou SQLite eussent été suffisants pour la V1, le choix de PostgreSQL est un choix d’ingénierie anticipatif. En effet, lors du passage au Produit Final (V2), le système nécessitera le stockage d’embeddings vectoriels (représentation mathématique du texte) pour le calcul de similarité sémantique. PostgreSQL, grâce à son extension **pgvector**, est actuellement le SGBD relationnel le plus robuste pour gérer nativement des bases de données vectorielles, nous évitant ainsi une migration coûteuse à l’avenir.

2.3.2 Environnement de Déploiement : Conteneurisation

Plutôt que d'installer la base de données directement sur le système d'exploitation de l'hôte, nous avons opté pour une approche par conteneurisation utilisant **Docker** et **Docker Compose**.

Preuves et avantages de ce choix :

- **Reproductibilité** : Tous les membres de l'équipe travaillent sur la même version exacte (PostgreSQL 15), éliminant le problème classique du "ça marche sur ma machine".
- **Isolation** : L'environnement de développement reste propre.
- **Infrastructure as Code** : La configuration est versionnée dans le fichier `docker-compose.yml`.

```
D:\2eme annee\S4\Projet\Base de donnees>docker compose up -d
time="2026-03-20T09:36:03Z" level=warning msg="D:\\2eme annee\\S4\\Projet\\Base de donnees\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 20/20
✔Image postgres:15                Pulled                20.7s
✔Network basedonnees_default      Created              0.2s
✔Volume basedonnees_postgres_data Created              0.1s
✔Container chatbot_postgres       Created              1.6s
```

FIGURE 6 – Configuration et exécution du conteneur PostgreSQL via Docker Compose

2.3.3 Modélisation des données

Pour la V1 basée sur les mots-clés, la solution de facilité aurait été de créer une seule table (**faqs**) contenant la question, la réponse et une colonne avec un mot-clé. Cependant, d'un point de vue architectural, cette approche n'est pas scalable. Un étudiant peut employer plusieurs synonymes pour la même requête (ex : "inscription", "réinscription").

Nous avons donc appliqué les règles de normalisation des bases de données en concevant une relation "Many-to-Many" (Plusieurs-à-Plusieurs) répartie sur trois tables :

1. **faqs** : Stocke l'identifiant, la question exacte et la réponse détaillée.
2. **keywords** : Stocke de manière unique les mots-clés de déclenchement.
3. **faq_keywords** : Table de jonction (ou table de liaison) associant un ID de FAQ à un ID de mot-clé avec des contraintes de clés étrangères (Foreign Keys).

Cette structure robuste permet d'associer un nombre illimité de mots-clés à une seule réponse sans redondance de données.

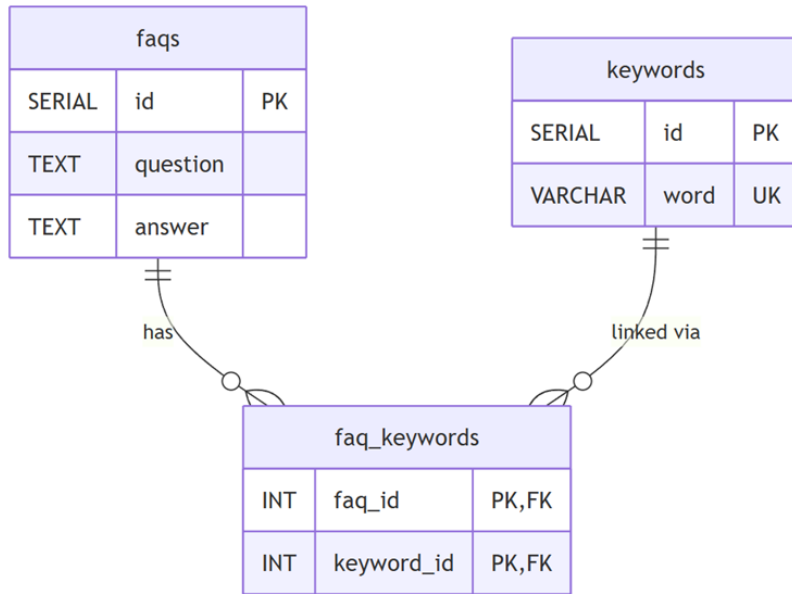


FIGURE 7 – Diagramme Entité-Relation (ER) de la base de données du prototype

2.3.4 Implémentation et Jeu de Données d'Essai

La création physique des tables a été réalisée directement via l'interface de commande de PostgreSQL (psql) exécutée à l'intérieur du conteneur Docker. Les contraintes ON DELETE CASCADE ont été ajoutées pour garantir l'intégrité référentielle : si une FAQ est supprimée, ses liaisons sont automatiquement purgées.

```

university_chatbot=# \dt
          List of relations
Schema |      Name      | Type  | Owner
-----+-----+-----+-----
public | faq_keywords   | table | chatbot_user
public | faqs           | table | chatbot_user
public | keywords       | table | chatbot_user
(3 rows)

```

FIGURE 8 – Vérification de la création des tables avec la commande \dt

Afin de tester notre backend, nous avons alimenté la base avec un jeu de données représentatif de la vie universitaire (scolarité, bourses, emploi du temps, bibliothèque).

```

1 -- Insertion des connaissances
2 INSERT INTO faqs (id, question, answer) VALUES
3 (4, 'Comment postuler pour la bourse Minhaty ?', 'La demande de bourse
   se fait en ligne...');
4
5 -- Insertion des déclencheurs
6 INSERT INTO keywords (id, word) VALUES (7, 'bourse'), (8, 'minhaty');
7
8 -- Création des liens (Le "cerveau" du prototype)
9 INSERT INTO faq_keywords (faq_id, keyword_id) VALUES (4, 7), (4, 8);

```

Listing 2 – Extrait de l'insertion des données et des liaisons

```

university_chatbot=# INSERT INTO faqs (id, question, answer) VALUES
university_chatbot=# (1, 'Comment obtenir une attestation de scolarité ?', 'Vous pouvez la demander au service de scolarité de l'EMI ou la télécharger via votre espace étudiant.'),
university_chatbot=# (2, 'Quand commencent les examens ?', 'Le calendrier des examens est affiché au département et sur le site web deux semaines avant le début des épreuves.'),
university_chatbot=# (3, 'Comment se réinscrire à l'université ?', 'La réinscription administrative se fait en ligne au début de chaque année universitaire.');
```

```

INSERT 0 3
university_chatbot=#
university_chatbot=# INSERT INTO keywords (id, word) VALUES
university_chatbot=# (1, 'attestation'),
university_chatbot=# (2, 'scolarite'),
university_chatbot=# (3, 'examen'),
university_chatbot=# (4, 'examens'),
university_chatbot=# (5, 'inscription'),
university_chatbot=# (6, 'reinscription');
```

```

INSERT 0 6
university_chatbot=#
university_chatbot=# INSERT INTO faq_keywords (faq_id, keyword_id) VALUES
university_chatbot=# (1, 1), (1, 2),
university_chatbot=# (2, 3), (2, 4),
university_chatbot=# (3, 5), (3, 6);
```

```

INSERT 0 6
university_chatbot=#

```

FIGURE 9 – Insertion des FAQs, mots-clés et liaisons dans PostgreSQL

2.3.5 Extraction des données pour le Backend

La conception normalisée impose l'utilisation de jointures SQL (JOIN) lors de l'interrogation par le backend Java. Lorsqu'un utilisateur saisit un terme, la requête suivante est exécutée pour traverser la table de liaison et récupérer la réponse exacte.

```

university_chatbot=# SELECT f.id, f.question, k.word
FROM faqs f
JOIN faq_keywords fk ON f.id = fk.faq_id
JOIN keywords k ON k.id = fk.keyword_id
ORDER BY f.id;
```

id	question	word
1	Comment obtenir une attestation de scolarité ?	attestation
1	Comment obtenir une attestation de scolarité ?	scolarite
2	Quand commencent les examens ?	examen
2	Quand commencent les examens ?	examens
3	Comment se réinscrire à l'université ?	inscription
3	Comment se réinscrire à l'université ?	reinscription
4	Comment postuler pour la bourse Minhaty ?	bourse
4	Comment postuler pour la bourse Minhaty ?	minhaty
5	Où puis-je consulter mon emploi du temps ?	emploi
5	Où puis-je consulter mon emploi du temps ?	temps
5	Où puis-je consulter mon emploi du temps ?	planning
6	Quelles sont les horaires d'ouverture de la bibliothèque ?	bibliotheque
6	Quelles sont les horaires d'ouverture de la bibliothèque ?	horaire
6	Quelles sont les horaires d'ouverture de la bibliothèque ?	horaires
7	Comment accéder aux services médicaux de l'université ?	medical
7	Comment accéder aux services médicaux de l'université ?	infirmierie
8	Quand est-ce que les résultats seront affichés ?	resultat
8	Quand est-ce que les résultats seront affichés ?	resultats

```

(18 rows)
university_chatbot=#

```

FIGURE 10 – Résultat de la requête de jointure validant le bon fonctionnement de la modélisation

2.4 Module Python : Préparation Anticipée de la Couche Intel- ligente

Bien que le prototype V1 repose sur un système déterministe de mots-clés géré par la Servlet Java, nous avons parallèlement développé un **microservice Python** autonome qui servira de moteur d'intelligence artificielle pour la V2. Ce choix permet de valider dès maintenant les composants NLP et de garantir une intégration fluide lors de l'évolution du système.

2.4.1 Architecture du Microservice

Le module Python est structuré selon une architecture modulaire en quatre fichiers distincts, chacun ayant une responsabilité claire :

1. `nlp_engine.py` : Le cœur du traitement linguistique (préprocessing et similarité sémantique).
2. `db.py` : La couche d'accès aux données (connexion PostgreSQL via `psycopg2`).
3. `app.py` : L'API REST exposée via le micro-framework **Flask**.
4. `populate_db.py` : Script utilitaire de peuplement initial de la base de données.

Cette séparation des responsabilités respecte le principe d'ingénierie logicielle *Separation of Concerns*, facilitant la maintenance et les tests unitaires de chaque composant.

2.4.2 Environnement et Dépendances

L'environnement Python est isolé dans un *virtual environment* (`.venv`) pour éviter les conflits de dépendances. Les bibliothèques utilisées sont déclarées dans un fichier `requirements.txt`, garantissant la reproductibilité de l'installation.

```
1 flask==3.0.3
2 psycopg2-binary>=2.9.0
3 spacy==3.7.4
4 scikit-learn==1.4.2
5 numpy==1.26.4
6 python-dotenv==1.0.1
```

Listing 3 – Fichier `requirements.txt` du module Python

Justification des choix de bibliothèques :

- **Flask** : Micro-framework léger pour exposer l'API REST. Il permet au backend Java de communiquer avec le moteur Python via de simples requêtes HTTP.
- **spaCy** : Bibliothèque industrielle de NLP offrant un pipeline complet (tokenisation, lemmatisation, suppression des *stop words*) avec un modèle pré-entraîné pour le français (`fr_core_news_sm`).
- **scikit-learn** : Fournit l'implémentation de TF-IDF et de la similarité cosinus, deux techniques fondamentales de la recherche d'information.
- **psycopg2** : Connecteur Python pour PostgreSQL, permettant au module de lire directement les FAQs depuis la même base de données que le prototype.
- **python-dotenv** : Gestion sécurisée des variables d'environnement (identifiants de la base de données) via un fichier `.env`.

2.4.3 Connexion à la Base de Données

Le module `db.py` gère la connexion à la base PostgreSQL de manière centralisée. Les paramètres de connexion sont externalisés dans un fichier `.env` afin de ne jamais exposer les identifiants dans le code source.

```
1 DB_USER=chatbot_user
2 DB_PASSWORD=mysecretpassword
3 DB_HOST=84.8.219.24
4 DB_PORT=5432
5 DB_NAME=university_chatbot
```

Listing 4 – Configuration de la connexion (`.env.example`)

```
1 import psycopg2
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 DB_USER      = os.getenv("DB_USER", "chatbot_user")
8 DB_PASSWORD  = os.getenv("DB_PASSWORD", "mysecretpassword")
9 DB_HOST      = os.getenv("DB_HOST", "84.8.219.24")
10 DB_PORT      = os.getenv("DB_PORT", "5432")
11 DB_NAME      = os.getenv("DB_NAME", "university_chatbot")
12
13 def get_connection():
14     conn = psycopg2.connect(
15         host=DB_HOST, port=DB_PORT,
16         dbname=DB_NAME, user=DB_USER,
17         password=DB_PASSWORD,
18         options="-c client_encoding=UTF8"
19     )
20     return conn
21
22 def get_all_faqs():
23     conn = get_connection()
24     cursor = conn.cursor()
25     try:
26         cursor.execute("SELECT question, answer FROM faqs")
27         rows = cursor.fetchall()
28         return [
29             {"question": row[0], "reponse": row[1]}
30             for row in rows
31         ]
32     finally:
33         cursor.close()
34         conn.close()
```

Listing 5 – Module de connexion à la base de données (`db.py`)

La fonction `get_all_faqs()` récupère l'ensemble des paires question/réponse depuis la table `faqs`. Le bloc `try/finally` garantit la fermeture systématique de la connexion, même en cas d'erreur, évitant ainsi les fuites de connexions (*connection leaks*).

2.4.4 Moteur NLP : Préprocessing et Similarité Sémantique

Le fichier `nlp_engine.py` constitue le composant le plus avancé du module. Il implémente un pipeline de traitement en deux étapes : la **normalisation linguistique** et la

recherche par similarité vectorielle.

Étape 1 : Préprocessing avec spaCy Avant toute comparaison, le texte brut de l'utilisateur est nettoyé et normalisé par la fonction `preprocess()`. Cette étape utilise le modèle linguistique français de spaCy pour :

- **Lemmatiser** chaque mot (réduire à sa forme canonique : "inscrire" au lieu de "inscriptions").
- **Supprimer les stop words** ("le", "de", "est"...) qui n'apportent pas de valeur sémantique.
- **Supprimer la ponctuation** et les caractères isolés.

```
1 import spacy
2
3 nlp = spacy.load("fr_core_news_sm")
4
5 def preprocess(text: str) -> str:
6     """
7     Nettoie et lemmatise un texte avec spaCy.
8     Ex: "Comment obtenir mes résultats ?"
9         -> "comment obtenir résultat"
10    """
11    doc = nlp(text.lower())
12    tokens = [
13        token.lemma_
14        for token in doc
15        if not token.is_stop
16        and not token.is_punct
17        and not token.is_space
18        and len(token.text) > 1
19    ]
20    return " ".join(tokens)
```

Listing 6 – Fonction de préprocessing linguistique (`nlp_engine.py`)

Étape 2 : Vectorisation TF-IDF et Similarité Cosinus La classe `NLPEngine` implémente la logique de recherche sémantique. Le principe est le suivant :

1. Au démarrage, toutes les questions de la FAQ sont **vectorisées** avec l'algorithme TF-IDF (*Term Frequency – Inverse Document Frequency*). Chaque question est transformée en un vecteur numérique dans un espace multidimensionnel.
2. Lorsqu'un utilisateur pose une question, celle-ci est vectorisée de la même manière.
3. On calcule la **similarité cosinus** entre le vecteur de la question et tous les vecteurs des FAQs. Un score de 1 indique une correspondance parfaite, 0 aucune similarité.
4. Si le meilleur score dépasse un **seuil configurable** (par défaut 0.3), la réponse associée est retournée.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3 import numpy as np
4
5 class NLPEngine:
6     def __init__(self, seuil_similarite: float = 0.3):
7         self.seuil = seuil_similarite
8         self.vectorizer = TfidfVectorizer()
```

```

9      self.faqs = []
10     self.questions_vec = None
11
12     def charger_faqs(self, faqs: list):
13         self.faqs = faqs
14         questions_traitees = [
15             preprocess(faq["question"]) for faq in faqs
16         ]
17         self.questions_vec = self.vectorizer.fit_transform(
18             questions_traitees
19         )
20         print(f"[NLP] {len(faqs)} FAQs vectorisées.")
21
22     def trouver_reponse(self, question_utilisateur: str) -> dict:
23         question_traitee = preprocess(question_utilisateur)
24         question_vec = self.vectorizer.transform(
25             [question_traitee]
26         )
27         scores = cosine_similarity(
28             question_vec, self.questions_vec
29         ).flatten()
30
31         meilleur_index = int(np.argmax(scores))
32         meilleur_score = float(scores[meilleur_index])
33
34         if meilleur_score >= self.seuil:
35             faq = self.faqs[meilleur_index]
36             return {
37                 "reponse": faq["reponse"],
38                 "score": round(meilleur_score, 4),
39                 "question_matchee": faq["question"],
40                 "trouve": True
41             }
42         else:
43             return {
44                 "reponse": "Je n'ai pas trouvé de réponse.",
45                 "score": round(meilleur_score, 4),
46                 "question_matchee": None,
47                 "trouve": False
48             }

```

Listing 7 – Classe NLPEngine – Recherche sémantique par TF-IDF

L'avantage majeur de cette approche par rapport au système de mots-clés de la V1 est sa capacité à comprendre des questions formulées différemment. Par exemple, la question « Je voudrais savoir comment m'inscrire » sera correctement associée à la FAQ « Comment s'inscrire à l'université ? », même si aucun mot-clé exact ne correspond.

2.4.5 API REST Flask

Le fichier `app.py` expose le moteur NLP sous forme d'API REST, permettant au backend Java de l'interroger via de simples requêtes HTTP. Trois endpoints sont définis :

- POST `/ask` : Reçoit une question en JSON et retourne la réponse la plus pertinente avec un score de confiance.
- POST `/reload` : Recharge les FAQs depuis la base de données sans redémarrage du serveur.

- GET /health : Vérifie l'état de santé du microservice (nombre de FAQs chargées, état du moteur).

```
1 from flask import Flask, request, jsonify
2 from nlp_engine import NLPEngine
3 from db import get_all_faqs
4
5 app = Flask(__name__)
6 engine = NLPEngine(seuil_similarite=0.3)
7
8 @app.route("/ask", methods=["POST"])
9 def ask():
10     data = request.get_json()
11     if not data or "question" not in data:
12         return jsonify(
13             {"erreur": "Le champ 'question' est requis."}
14         ), 400
15
16     question = data["question"].strip()
17     if not question:
18         return jsonify(
19             {"erreur": "La question ne peut pas être vide."}
20         ), 400
21
22     resultat = engine.trouver_reponse(question)
23     return jsonify(resultat), 200
```

Listing 8 – API REST Flask (app.py) – Endpoint principal

Mécanisme de résilience : Au démarrage, l'application tente de charger les FAQs depuis PostgreSQL. En cas d'échec de connexion (serveur indisponible, réseau coupé), un mécanisme de *fallback* charge automatiquement les données depuis un fichier JSON local (data/faq_universitaire.json), garantissant ainsi le fonctionnement du moteur même en mode dégradé.

```
1 def initialiser_moteur():
2     try:
3         faqs = get_all_faqs()
4         engine.charger_faqs(faqs)
5     except Exception as e:
6         print(f"[ERREUR] Impossible de charger : {e}")
7         _charger_faqs_fallback()
8
9 def _charger_faqs_fallback():
10     import json
11     with open("data/faq_universitaire.json", "r",
12             encoding="utf-8") as f:
13         faqs = json.load(f)
14     engine.charger_faqs(faqs)
```

Listing 9 – Mécanisme de fallback au démarrage

2.4.6 Jeu de Données de Secours (JSON)

Le fichier data/faq_universitaire.json contient un jeu de 10 FAQs couvrant les thématiques universitaires essentielles. Ce fichier sert à la fois de données de fallback et de référence pour les tests du moteur NLP.


```

1 [
2   {
3     "keyword": "inscription",
4     "question": "Comment s'inscrire à l'université ?",
5     "reponse": "Pour vous inscrire, rendez-vous sur
6                 le portail étudiant avec votre
7                 baccalauréat et pièce d'identité.
8                 Les inscriptions ouvrent en juillet."
9   },
10  {
11    "keyword": "bourse",
12    "question": "Comment obtenir une bourse ?",
13    "reponse": "Les demandes de bourses se font en
14               ligne sur le portail du CNOPS avant
15               le 30 septembre..."
16  },
17  {
18    "keyword": "examen",
19    "question": "Quand ont lieu les examens ?",
20    "reponse": "Les examens de la session normale se
21               déroulent en janvier pour le S1 et en
22               juin pour le S2..."
23  }
24 ]

```

Listing 10 – Extrait du jeu de données (faq_universitaire.json)

2.4.7 Script de Peuplement de la Base de Données

Le script `populate_db.py` automatise la création de la table et l'insertion des FAQs en base de données à partir du fichier JSON. Il intègre une vérification d'existence de la table avant création, évitant les erreurs lors d'exécutions successives.

```

1 def creer_table(cursor):
2     """Crée la table FAQ si elle n'existe pas."""
3     if table_existe(cursor, "faq"):
4         print("[INFO] La table FAQ existe déjà.")
5     else:
6         cursor.execute(SQL_CREATE_TABLE)
7         print("[OK] Table FAQ créée.")
8
9 def inserer_faqs(cursor, faqs: list):
10    """Insère toutes les FAQs dans la table."""
11    nb_inseres = 0
12    for faq in faqs:
13        cursor.execute(SQL_INSERT, {
14            "keyword": faq["keyword"],
15            "question": faq["question"],
16            "reponse": faq["reponse"]
17        })
18        nb_inseres += 1
19    print(f"[OK] {nb_inseres} FAQs insérées.")

```

Listing 11 – Extrait du script de peuplement (`populate_db.py`)

2.4.8 Synthèse et Rôle dans la V2

Le développement anticipé de ce module Python constitue un investissement stratégique pour le projet. Lors du passage à la V2, l'intégration se fera de manière transparente :

1. Le **backend Java** redirigera les questions de l'utilisateur vers l'API Flask (POST `/ask`) au lieu de faire une recherche par mots-clés en base de données.
2. Le moteur NLP retournera la réponse la plus pertinente avec un **score de confiance**, permettant au frontend d'adapter son affichage (ex : afficher un avertissement si le score est faible).
3. L'endpoint `/reload` permettra de mettre à jour les connaissances du chatbot sans interruption de service.

Ce choix architectural de microservice découplé (*loosely coupled*) offre également la possibilité de faire évoluer indépendamment le moteur NLP (passage de TF-IDF à des modèles de langage plus avancés comme BERT ou Sentence-Transformers) sans impacter le reste du système.